ack3

SECURITY PARTNER

Safe

Policy Engine

ENGAGEMENT	SECURITY REVIEW
REVISION	1.1
DELIVERED	2026-11-09
LEAD AUDITOR	Michal Převrátil

Contents

1. Document Revisions . . . 3
2. Overview . . . 4
  2.1. ack3 . . . 4
  2.2. Review Methodology . . . 5
  2.3. Finding Classification . . . 6
  2.4. Review Team . . . 8
  2.5. Disclaimer . . . 8
3. Executive Summary . . . 9
  Revision 1.0 . . . 9
  Revision 1.1 . . . 10
4. Findings Summary . . . 12
Report Revision 1.0 . . . 15
  Revision Team . . . 15
  System Overview . . . 15
  Trust Model . . . 15
  Findings . . . 16
Report Revision 1.1 . . . 64
  Revision Team . . . 64
  System Overview . . . 64
  Trust Model . . . 64
Appendix A: How to cite . . . 65

1. Document Revisions

1.0-draft	Draft Report	September 25, 2026
1.1	Fix Review	November 9, 2026

2. Overview

This document presents the findings of our review.

2.1. ack3

ack3 is an in-house team of security researchers performing audits focused on manual code reviews with extensive fuzz testing and AI-assisted workflows for Ethereum and Solana. The ack3 team is trusted by top-tier organizations in web3, including Lido, Safe, and Axelar.

We develop the open-source security and developer tooling Wake, which powers the fuzzing, static analysis, and graph-based review on our audits and helps auditors discover hardly recognizable edge-case vulnerabilities.

Our team teaches blockchain security at the Czech Technical University in Prague, led by founder Josef Gattermayer, Ph.D.

For ack3, security is a long-term partnership: we work alongside our clients from design through post-deployment.

2.2. Review Methodology

We begin each review by mapping the codebase and the interactions between contracts and external components using Wake's graph tooling. We work with the provided documentation and materials to discover discrepancies between the expected behavior and the actual implementation. Where applicable, we build a mathematical model of the protocol to cross-compare with the code and identify subtle bugs and inconsistencies.

At the start of the review, we launch an automated ack3 AI scan pinned to the scoped commits, running in parallel with the manual review. The scan provides additional coverage and new leads for the team to analyze; each scan finding is manually reviewed by the team.

During the review, we apply AI-assisted workflows and skills that support codebase onboarding and navigation, combined with Wake's static analyzer and language server for context-aware code analysis.

We launch AI agents to build on our vulnerability hypotheses and expand the attack surface with new escalation paths. All findings are manually reviewed and validated by the team. Where applicable, we confirm findings and verify the exact behavior of the code with unit tests and fuzzing. Where the engagement includes a fuzzing campaign, it is manually guided by an auditor; the fuzz tests themselves are AI-generated.

Once sufficient context is gathered, we search for deeper, chained issues based on the attack vectors discovered during review. This includes combining smaller weaknesses, testing trust-boundary assumptions, and evaluating whether unexpected behavior can lead to practical impact.

The validated results are documented in the report: findings with severity, affected components, impact, and recommended remediation.

2.3. Finding Classification

The Severity of each finding combines two sub-ratings, Impact and Likelihood, and ranges from Info to Critical.

If we find a scenario in which an issue is exploitable, it is assigned an impact rating of High, Medium, or Low, based on its consequences for the system. If we find no way to exploit it, or the issue is only exploitable given a change in configuration (system settings or parameters, such as deployment scripts, compiler configurations, or multi-signature wallets for owners) or in the codebase, it is assigned Warning or Info.

Low to High impact issues also have a Likelihood, which measures the probability of exploitation at runtime.

Severity

		LIKELIHOOD			
		HIGH	**MEDIUM**	**LOW**	**N/A**
IMPACT	**HIGH**	Critical	High	Medium	-
	MEDIUM	High	Medium	Low	-
	LOW	Medium	Low	Low	-
	WARNING	-	-	-	Warning
	INFO	-	-	-	Info

Table 1. Severity of findings

Impact

- **High** - Code that activates the issue leads to undefined or catastrophic consequences for the system.
- **Medium** - Code that activates the issue results in serious consequences.
- **Low** - Code that activates the issue has consequences that are recoverable or do not jeopardize the system's normal operation.
- **Warning** - The issue cannot be exploited given the current code or `configuration`, but could become a security vulnerability if either changed. If we have not found a way to exploit the issue given the time constraints, it may be marked as `Warning` or higher, based on our best estimate of whether it is currently exploitable.
- **Info** - The issue is on the borderline between code quality and security. Examples include insufficient logging for critical operations, or issues that would become security-related if code or `configuration` changed.

Likelihood

- **High** - The issue is exploitable by virtually anyone under any circumstance.
- **Medium** - Exploiting the issue requires non-trivial preconditions.
- **Low** - Exploiting the issue requires strict preconditions.

2.4. Review Team

For a revision's authors, see the "Revision team" section in that revision's chapter.

NAME	POSITION
Michal Převrátil	Lead Auditor
Josef Gattermayer, Ph.D.	Audit Supervisor

2.5. Disclaimer

We have made our best effort to find all vulnerabilities in the system; however, our findings should not be considered a complete list of issues. Statements in this document should not be interpreted as investment or legal advice, nor should the authors be held accountable for decisions based on them.

3. Executive Summary

Safe Policy Engine is a set of rules that controls what a Safe crypto wallet is allowed to do. For example, it can allow token transfers only to approved recipients or require extra signatures for certain actions. Transactions are blocked unless the rules allow them, even if the wallet's usual signers have already approved them.

Revision 1.0

Safe engaged ack3 to perform a security review of Safe Policy Engine with a total time donation of 6.5 engineering days in a period between September 14 and September 25, 2026, with Michal Převrátil as the lead auditor.

The review covered commit `405ba1d`[1] of the policy-engine repository and included all files under `contracts` except `contracts/test`, `contracts/policies/IncreasedThresholdPolicy.sol`, and `contracts/policies/OneTimeAllowPolicy.sol`.

We manually reviewed the in-scope code and ran an automated AI scan focused on the in-scope code and its perimeter: contracts that interact with the in-scope code and off-chain code directly related to it. We also prepared targeted, manually guided fuzz tests in Wake for the cryptography in `FROST.sol` and interactions with the Safe wallet. We manually verified the scan results, included valid in-scope findings in the report, and sent out-of-scope findings directly to the client.

We focused on whether:

- the cryptography used for the Safenet integration was sound and followed RFC-9591;
- the policies followed defense-in-depth practices without being overly permissive;

- the Policy Engine introduced pitfalls that could cause users to make mistakes;
- any remaining pitfalls were well documented.

Our review resulted in 22 findings, ranging from Info to Medium severity. The most severe findings involved two gaps in defense in depth: module transactions permitted to change guard settings could remove the Safe guards that enforce the Policy Engine (M2), and `NativeTransferPolicy` could approve calls with nonzero value and arbitrary calldata when configured as a fallback (M1).

ack3 recommends Safe:

- synchronize the helper libraries with the safenet upstream;
- document edge cases in the cryptographic libraries;
- fix or clearly document the counterintuitive Policy Engine behaviors identified in the findings.

See Report Revision 1.0 for the system overview and trust model.

Revision 1.1

Safe engaged ack3 to perform a fix review of the findings from the previous revision.

Safe provided fixes and comments to the acknowledged findings.

The review was performed between November 5 and November 7, 2026 on the commit 8ee68a8[2]. We compared that commit with the previous revision's scoped commit and examined the in-scope changes from PR #167, PR #168, PR #170, PR #171, PR #172, and PR #174. PR #106 and PR #169 changed files outside the reviewed `contracts` scope.

From the reported 22 findings:

- Fourteen findings were fixed, including M1, W5, and W7; and
- Eight findings were acknowledged, including M2 and L1.

No new findings were reported.

See Report Revision 1.1 for more details.

[1] full commit hash: `405ba1d91e85d0fd26abf0f645bf216822c8c595`, link to commit

[2] full commit hash: `8ee68a8cdac65b4aba5d9a10e9f9018c2efb10b3`, link to commit

4. Findings Summary

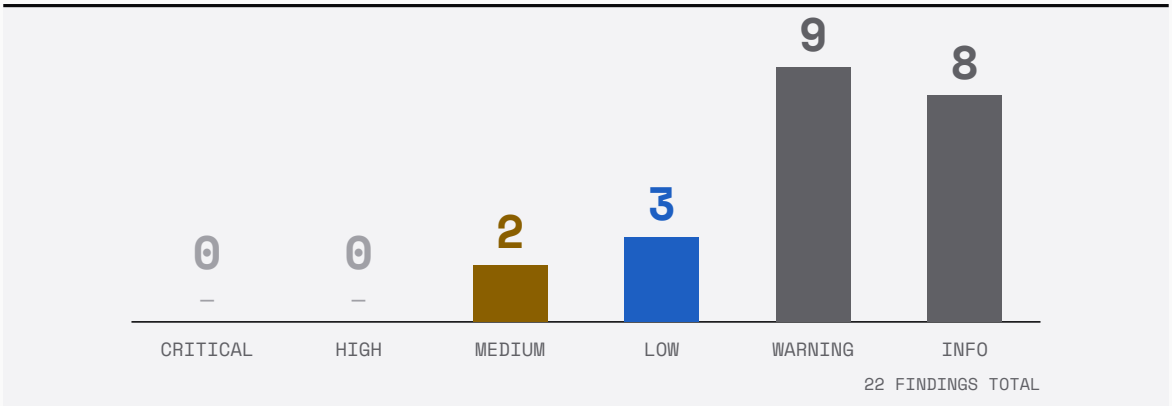


Figure 1. Findings by Severity

FINDING TITLE	SEVERITY	REVISION	STATUS
M1: Calldata-Blind Native Transfer Fallback	MEDIUM	1.0	Fixed
M2: Missing module-execution guard preservation	MEDIUM	1.0	Acknowledged
L1: Missing expiry for Safenet transaction attestations	LOW	1.0	Acknowledged
L2: Co-signer getter namespace mismatch	LOW	1.0	Fixed
L3: Missing module path rejection in CoSignerPolicy	LOW	1.0	Fixed
W1: Missing misuse warning for FROST.nonce	WARNING	1.0	Fixed

FINDING TITLE	SEVERITY	REVISION	STATUS
W2: Incorrect FROST XMD length limit	WARNING	1.0	Fixed
W3: Unauthenticated FROST Share Inputs	WARNING	1.0	Acknowledged
W4: Secp256k1 identity addition memory aliasing	WARNING	1.0	Fixed
W5: Missing zero-divisor rejection in `_divmod`	WARNING	1.0	Fixed
W6: Incomplete gas field validation for Safe hash reconstruction	WARNING	1.0	Acknowledged
W7: Selector-zero access key ambiguity	WARNING	1.0	Fixed
W8: Unbounded epoch rollover	WARNING	1.0	Acknowledged
W9: Retained policy grants after detachment	WARNING	1.0	Acknowledged
I1: Unsupported high-x group keys	INFO	1.0	Acknowledged
I2: Unused `InvalidSelector` error	INFO	1.0	Fixed
I3: Missing nonzero FROST identifier check	INFO	1.0	Fixed

FINDING TITLE	SEVERITY	REVISION	STATUS
I4: Inaccurate NatSpec comments	INFO	1.0	Fixed
I5: Undocumented co-signature format differences	INFO	1.0	Fixed
I6: Incorrect guard-removal documentation and test	INFO	1.0	Fixed
I7: Fallback authorization after exact policy clearing	INFO	1.0	Fixed
I8: Ambiguous policy replacement events	INFO	1.0	Acknowledged

Table 2. Table of Findings

Report Revision 1.0

Revision Team

NAME	POSITION
Michal Převrátil	Lead Auditor
Josef Gattermayer, Ph.D.	Audit Supervisor

System Overview

Safe Policy Engine adds configurable transaction rules to Safe accounts. `SafePolicyGuard` acts as both a transaction guard and a module guard, routing owner-approved and module-executed transactions through `PolicyEngine`. For each Safe, `PolicyEngine` selects a policy contract by target address, function selector, and operation. If no exact policy is configured, it uses the operation-wide fallback if one exists; otherwise, it denies ordinary transactions. The supplied policies cover native value transfers, ERC-20 transfers and approvals, module access, batched transactions, additional signatures, and Safenet FROST attestations. A Safe can configure policies immediately before installing the guard; subsequent changes are requested and applied after the guard's configured delay.

Trust Model

Safe owners choose the guard and policy configuration and can authorize fund transfers that satisfy the active policies. Policy enforcement requires `SafePolicyGuard` to be installed as both the transaction guard and module guard; an unguarded path can move funds without policy checks. Enabled modules can initiate transactions without owner signatures and move Safe funds when an active policy permits the call. Each configured policy contract

is trusted to enforce its intended limits, since a permissive or faulty policy can approve unintended transfers. When configured, a co-signer or the Safenet FROST signer group can approve or withhold approval for covered transactions, but their approval alone cannot move funds. Policy changes after guard installation are subject to the guard's configured delay.

Findings

This section lists the findings from this revision; for all findings, see the Findings Summary.

M1: Calldata-Blind Native Transfer Fallback

MEDIUM SEVERITY ISSUE

Impact:	High	**Likelihood:**	Low
Type:	Data validation	**Discoverer:**	AI
Target:	NativeTransferPolicy.sol, PolicyEngine.sol		

Description

`NativeTransferPolicy` can be configured for the operation-wide `CALL` fallback because `configure` accepts a zero selector.

Listing 1. Excerpt from NativeTransferPolicy

```
36 function configure(address, AccessSelector.T access, bytes memory) external
   pure override returns (bool) {
37     return access.getSelector() == bytes4(0) && access.getOperation() ==
   Operation.CALL;
```

When no exact policy exists for the actual target and selector, `PolicyEngine.getPolicy` replaces the exact access key with the fallback key.

Listing 2. Excerpt from PolicyEngine.getPolicy

```
134 // Use the fallback policy for the given operation if there is no specific
    one.
135 if (policy == address(0)) {
136     access = AccessSelector.createFallback(operation);
137     policy = policies[access];
138 }
```

`NativeTransferPolicy.checkTransaction` then approves any positive-value transaction and ignores the actual calldata.

Listing 3. Excerpt from NativeTransferPolicy.checkTransaction

```
22      uint256 value,
23      bytes calldata,
24      Operation,
25      address,
26      bytes calldata,
27      AccessSelector.T
28  ) external pure override returns (bytes4 magicValue) {
29      require(value > 0, InvalidTransfer());
30      return IPolicy.checkTransaction.selector;
```

As a result, a value-bearing function call with well-formed calldata can be authorized by the native-transfer fallback whenever no stricter exact policy is configured for that selector. The callee observes the Safe as `msg.sender`, so the call can use standing approvals or other Safe authority and move non-native assets outside the intended native-transfer permission.

Exploit scenario

Alice configures `NativeTransferPolicy` as the `CALL` fallback to allow plain native transfers from her Safe. The Safe also has an existing token approval to a payable integration.

Bob obtains the required module or owner authorization for a positive-value call to that integration with calldata for a payable function that pulls tokens from `msg.sender`. Because no exact policy is configured for that selector, the engine selects the native-transfer fallback. `NativeTransferPolicy` approves the transaction based only on the positive value, and the integration pulls tokens from Alice's Safe.

Recommendation

If `NativeTransferPolicy` is intended to allow only plain native transfers, validate the actual runtime tuple in `checkTransaction`: require `Operation.CALL`, positive `value`, and `data.length == 0`. Put payable protocol interactions behind separate policies that validate the target, selector, arguments, and

relevant authority.

Fix 1.1

`NativeTransferPolicy` now requires `Operation.CALL` and empty transaction calldata, and rejects nonempty configuration data.

Go back to Findings Summary

M2: Missing module-execution guard preservation

MEDIUM SEVERITY ISSUE

Impact:	High	**Likelihood:**	Low
Type:	Access control	**Discoverer:**	AI
Target:	SafePolicyGuard.sol		

Description

`SafePolicyGuard` does not preserve the Safe guard configuration across module execution. `checkModuleTransaction` runs the selected policy and exits before execution without recording the current transaction guard or module guard.

Listing 4. Excerpt from SafePolicyGuard.checkModuleTransaction

```
255 _enterCheck(msg.sender, module);
256 checkTransaction(msg.sender, to, value, data, operation, _emptyContext());
257 _exitCheck();
```

The post-hook only verifies that the module transaction succeeded. It does not reject successful module transactions that changed either guard slot.

Listing 5. Excerpt from SafePolicyGuard

```
268 function checkAfterModuleExecution(bytes32, bool success) external pure
    override {
269     require(success, ModuleExecutionFailed());
```

As a result, a policy that authorizes a module call to `setGuard` or `setModuleGuard` lets the module remove policy enforcement for the corresponding execution path. This includes broad module authorization and explicit `AllowPolicy` entries for guard removal, because module and owner transactions use the same policy resolution boundary.

Exploit scenario

Alice's Safe has `SafePolicyGuard` installed as both guards and has an enabled module that Mallory can direct. A configured policy allows that module to perform Safe administration, or explicitly allows `setGuard` or `setModuleGuard`. Mallory executes the guard-removal call through the module. The pre-hook accepts it, the Safe updates the guard slot, and `checkAfterModuleExecution` accepts the transaction because it only checks `success`. Mallory then uses the unguarded execution path to bypass the policy engine.

Recommendation

Capture both Safe guard addresses in `checkModuleTransaction` and carry that state to the matching post-hook. In `checkAfterModuleExecution`, require success and require both guard addresses to match the captured values, then clear the stored context. If module-authorized guard changes are intended, encode them as explicit expected transitions rather than allowing generic module policy approval to remove enforcement.

Acknowledgment 1.1

The finding was acknowledged and deferred to a future iteration.

Go back to Findings Summary

L1: Missing expiry for Safenet transaction attestations

LOW SEVERITY ISSUE

Impact:	Low	**Likelihood:**	Low
Type:	Logic error	**Discoverer:**	AI
Target:	ConsensusMessages.sol, SafenetPolicy.sol		

Description

The signed `TransactionProposal` message binds the validator epoch, oracle, oracle data, and Safe transaction hash, but it does not include a deadline or other freshness field.

Listing 6. Excerpt from ConsensusMessages

```
30  * @custom:precomputed keccak256("TransactionProposal(uint64 epoch,address
    oracle,bytes oracleData,bytes32 safeTxHash)")
31  */
32 bytes32 internal constant TRANSACTION_PROPOSAL_TYPEHASH =
33     hex"9c6706f5afdb1de99f5ad39011e7770ce471f51d78380634f6cedb21a648b8d0";
```

`SafenetPolicy.checkTransaction` verifies that the attestation key is known, that the Safe nonce-bound transaction hash matches, and that the FROST signature is valid. It only prevents reuse after consumption, so an unspent attestation can remain executable indefinitely while the Safe nonce stays current and the transaction still routes through this policy.

Listing 7. Excerpt from SafenetPolicy

```
123 // Cheap membership check first, so an untrusted key short-circuits before
    the hashing.
124 require($epochs.isKnown(groupKey, epoch), UntrustedAttestationKey());
125
126 // The Safe increments its nonce before invoking the guard.
127 uint256 nonce = ISafe(safe).nonce() - 1;
```

```
128 require(!$spent[msg.sender][safe][nonce], AttestationAlreadySpent());
```

Listing 8. Excerpt from SafenetPolicy

```
145 bytes32 message = ConsensusMessages.transactionProposal(
146     _CONSENSUS_DOMAIN_SEPARATOR,
147     epoch,
148     oracle,
149     oracleDataHash,
150     safeTxHash
151 );
152 FROST.verify(groupKey, signature, message);
```

As a result, time-sensitive oracle assessments can become stale. A transaction approved when its destination was acceptable can still pass later even if a fresh oracle decision would reject it.

Exploit scenario

Alice's Safe receives a valid Safenet attestation for a transaction to Bob's contract while Bob's contract is considered acceptable. Alice does not execute the transaction, and the Safe nonce remains unchanged. Bob's contract is later classified as malicious. Alice or a relayer submits the old Safe transaction and attestation; `checkTransaction` accepts the unspent signature because no signed deadline is enforced.

Recommendation

Add a signed execution deadline, such as `validUntil`, to `TransactionProposal` and enforce it in `SafenetPolicy.checkTransaction`. Coordinate the message schema, validator signing flow, attestation encoding, and verifier logic so the deadline is covered by the validator signature. If immediate withdrawal of existing approvals is required, add an authenticated revocation or authorization-version mechanism.

Acknowledgment 1.1

The finding was acknowledged.

Go back to Findings Summary

L2: Co-signer getter namespace mismatch

LOW SEVERITY ISSUE

Impact:	Low	**Likelihood:**	Medium
Type:	Logic error	**Discoverer:**	AI
Target:	CoSignerPolicy.sol		

Description

`CoSignerPolicy.configure` stores co-signers under the caller namespace, which is the `SafePolicyGuard` when configuration is applied through the guard.

Listing 9. Excerpt from CoSignerPolicy.configure

```
86 $cosigners[msg.sender][safe][access] = cosigner;
```

`CoSignerPolicy.getCoSigner` also reads the caller namespace, although ordinary callers are wallets, dashboards, integrations, or other on-chain consumers rather than the guard. As a result, the getter returns `address(0)` or an unrelated caller-local configuration instead of the co-signer used by policy enforcement.

Listing 10. Excerpt from CoSignerPolicy

```
96 function getCoSigner(address safe, AccessSelector.T access) external view
   returns (address cosigner) {
97     cosigner = $cosigners[msg.sender][safe][access];
```

Exploit scenario

Alice configures a co-signer through `SafePolicyGuard`. Her monitoring integration later calls `getCoSigner` from its own address to verify the

configuration. The getter reads the integration namespace, so it reports no co-signer or a different co-signer, causing monitoring and dependent consumers to act on incorrect configuration data while enforcement still uses the guard namespace.

Recommendation

Add an explicit `policyGuard` argument to `getCoSigner` and read `$cosigners[policyGuard][safe][access]`. Update the NatSpec and callers accordingly.

Fix 1.1

The finding was fixed by following the recommendation.

Go back to Findings Summary

L3: Missing module path rejection in CoSignerPolicy

LOW SEVERITY ISSUE

Impact:	Low	**Likelihood:**	Low
Type:	Access control	**Discoverer:**	Human
Target:	CoSignerPolicy.sol		

Description

`CoSignerPolicy.checkTransaction` discards the authenticated `module` parameter instead of rejecting module executions. The same check then rebuilds a Safe transaction hash with `ISafe(safe).nonce() - 1`, which is only valid for owner transactions because module executions do not increment the Safe nonce.

Listing 11. Excerpt from CoSignerPolicy.checkTransaction

```
42 function checkTransaction(
43     address safe,
44     address to,
45     uint256 value,
46     bytes calldata data,
47     Operation operation,
48     address,
49     bytes calldata context,
```

Listing 12. Excerpt from CoSignerPolicy.checkTransaction

```
63 address(0),
64 address(0),
65 ISafe(safe).nonce() - 1 // The Guard check is executed post nonce
   increment, so we need to subtract 1 from the nonce.
```

On the module path, this usually fails closed. However, with a contract co-signer that accepts an empty ERC-1271 signature for a pre-approved hash,

the policy can validate a stale, unspent approval for the previous owner nonce. An enabled module that reaches the same access selector can therefore execute that exact stale transaction without a fresh co-signature.

Listing 13. Excerpt from CoSignerPolicy.checkTransaction

```
72 bool validSignature = SignatureChecker.isValidSignatureNow(cosigner,
   safeTxHash, context);
73 require(validSignature, Unauthorized());
```

Exploit scenario

Alice's Safe uses `CoSignerPolicy` for an access selector that an enabled module can also reach, and its co-signer is another Safe. The co-signer Safe pre-approves transaction `T`, but Alice's Safe executes a different owner transaction at that nonce, leaving the approval unspent. Bob controls the enabled module and later submits `T` with the same `to`, `value`, `data`, and `operation`. `CoSignerPolicy` ignores the module path, reconstructs the stale hash with `nonce() - 1`, accepts the ERC-1271 pre-approval, and allows the module transaction without a fresh co-signature.

Recommendation

Name the `module` parameter and reject module transactions at the start of `checkTransaction` with the same `ModulePathUnsupported()` check used by the sibling nonce-pinning policies. Update the NatSpec accordingly.

Fix 1.1

The finding was fixed by following the recommendation.

Go back to Findings Summary

W1: Missing misuse warning for FROST.nonce

Impact:	Warning	**Likelihood:**	N/A
Type:	Code quality	**Discoverer:**	AI
Target:	FROST.sol		

Description

The vendored `FROST.nonce` NatSpec describes nonce generation from `random` and a participant secret, but omits the upstream warning that the helper is only for reference/testing use and must not be called with real signing shares through transactions or remote RPC. Passing a real share to this helper exposes it publicly in transaction calldata, or to the RPC provider during `eth_call`.

Listing 14. Excerpt from FROST.nonce

```
84 * @notice Generates a random nonce from randomness and a secret key.
85 * @param random The source randomness.
86 * @param secret The participant secret key.
87 * @return n The derived nonce scalar.
88 * @dev Implements the RFC-9591 `nonce_generate` function
89 *      (https://datatracker.ietf.org/doc/html/rfc9591#section-4.1).
```

The hash derivation is not the issue; callers remain responsible for fresh cryptographically secure randomness and one-time nonce use. No production caller was identified in scope, so the risk is integration misuse by downstream callers relying on the vendored documentation.

Recommendation

Mark `FROST.nonce` as reference/testing-only, and state that real signers must generate nonces locally with fresh cryptographically secure randomness, use each nonce once, and never transmit secret signing shares to contracts or

remote RPC endpoints.

Fix 1.1

The `FROST.nonce` NatSpec now warns against passing real signing shares in transactions or RPC calls.

Go back to Findings Summary

W2: Incorrect FROST XMD length limit

Impact:	Warning	**Likelihood:**	N/A
Type:	Standards violation	**Discoverer:**	AI
Target:	FROST.sol		

Description

`FROST._expandMessageXmd` accepts `len < 0x8000`, but RFC 9380 `expand_message_xmd` must reject requests where `ceil(len / b_in_bytes) > 255`. For SHA-256, `b_in_bytes` is `32`, so the maximum valid output length is `8160` bytes.

Listing 15. Excerpt from FROST._expandMessageXmd

```
372 assert(len < 0x8000);
```

The block counter is written with `mstore8`, so requests above `8160` bytes require block index `256` and truncate the index to one byte instead of reverting. The resulting expansion is outside the RFC-defined algorithm.

Listing 16. Excerpt from FROST._expandMessageXmd

```
414 for { let i := 2 } gt(len, 0x20) {
415     i := add(i, 1)
416     len := sub(len, 32)
417 } {
418     let uPtr := add(uniform, shl(5, i))
419     mstore(bPtr, xor(b0, mload(sub(uPtr, 0x20))))
420     mstore8(iPtr, i)
```

This is not currently reachable as a production failure because `_expandMessageXmd` is private and its only caller requests `48` bytes. The risk is future reuse of the helper with larger output lengths.

Recommendation

Replace the bound with `assert(len <= 8160)` and document that the SHA-256 XMD maximum is `255 * 32` bytes. Import the upstream Safenet fix through the vendoring process and keep the recorded source revision consistent with `vendor:check`.

Fix 1.1

The XMD length assertion now enforces the 8,160-byte limit.

Go back to Findings Summary

W3: Unauthenticated FROST Share Inputs

Impact:	Warning	**Likelihood:**	N/A
Type:	Data validation	**Discoverer:**	AI
Target:	FROST.sol, Secp256k1.sol		

Description

`FROST.verifyShare` accepts the participant commitment `share.r`, the aggregate commitment `r`, and the Lagrange coefficient `share.l` from its caller instead of deriving and authenticating them from the FROST signing transcript and signer set. As a result, the function only checks that the supplied values satisfy a self-consistent equation, and an integration that treats a successful return as proof of an authentic protocol share can misattribute or accept invalid share data.

Listing 17. Excerpt from FROST.verifyShare

```
182 uint256 c = challenge(r, group, message);
183 uint256 cl = mulmod(c, share.l, Secp256k1.N);
184 Secp256k1.mulmuladd(share.z, cl, participant, share.r);
```

The equation helper also compares the `ecrecover` result against a 160-bit address digest of the supplied witness point, not the full point. This truncation is documented as an implementation caveat, but the API does not state the lower generic birthday bound when both compared values can vary independently.

Listing 18. Excerpt from Secp256k1.mulmuladd

```
158 mstore(0x00, rx)
159 mstore(0x20, sub(P, ry))
160 valid := eq(minusR, and(keccak256(0x00, 0x40),
    0xffffffffffffffffffffffffffffffffffffffff))
```

This is a library API hardening issue. There is no production caller of `FROST.verifyShare` in this repository, and production signature checks use `FROST.verify`, where the same aggregate commitment is bound into the challenge and used as the witness.

Recommendation

Update `FROST.verifyShare` to consume authenticated nonce commitments and the signer list, then derive the commitment share, aggregate commitment, binding factor, and interpolation coefficient according to RFC 9591. Alternatively, document that it is only an equation-checking helper and that callers must independently derive and authenticate all inputs before relying on the result.

Document that `Secp256k1.mulmuladd` checks equality of 160-bit address digests and has a lower generic collision bound when both inputs vary independently. Use a full point-relation check for integrations that require stronger equality guarantees.

Acknowledgment 1.1

The finding was acknowledged.

Go back to Findings Summary

W4: Secp256k1 identity addition memory aliasing

Impact:	Warning	**Likelihood:**	N/A
Type:	Code quality	**Discoverer:**	AI
Target:	Secp256k1.sol		

Description

`Secp256k1.add` returns `q` or `p` directly when the other operand is the point at infinity. Because `Point` is a memory struct, an internal caller receives an alias to the original operand in these branches, while other branches populate the named return value.

Listing 19. Excerpt from Secp256k1.add

```
86 if (px | py == 0) {
87     // `P` is the point at infinity, and `0 + Q = Q`.
88     return q;
89 } else if (qx | qy == 0) {
90     // `Q` is the point at infinity, and `P + 0 = P`.
91     return p;
```

Future callers that mutate the returned `Point` could unexpectedly mutate their input. There are currently no production callers, so this is a hardening issue.

Recommendation

Document the aliasing behavior in the NatSpec, or return a fresh `Point` by copying the surviving operand's coordinates into `r` in both identity branches.

Fix 1.1

`Secp256k1.add` now copies the surviving point's coordinates into the return value.

Go back to Findings Summary

W5: Missing zero-divisor rejection in `_divmod`

Impact:	Warning	**Likelihood:**	N/A
Type:	Data validation	**Discoverer:**	AI
Target:	Secp256k1.sol		

Description

`Secp256k1._divmod` treats a successful `MODEXP` call as a valid field inverse. For `y == 0`, `MODEXP` succeeds and returns zero modulo `P`; `_divmod` then returns `0` instead of rejecting undefined division.

Listing 20. Excerpt from Secp256k1._divmod

```
259 success := and(eq(returndatasize(), 0x20), staticcall(gas(), 0x5, ptr,
    0xc0, 0x00, 0x20))
```

Listing 21. Excerpt from Secp256k1._divmod

```
262     result := mulmod(x, mload(0x00), p)
263 }
264 assert(success);
```

The current `add` callers make zero divisors unreachable under their branch and point-validity assumptions, and `add` has no production callers. The issue is a hardening gap if those assumptions change.

Recommendation

Reject `y == 0` in `_divmod` and document that the divisor must be a nonzero field element.

Fix 1.1

The vendored `Secp256k1._divmod` now rejects a zero divisor.

Go back to Findings Summary

W6: Incomplete gas field validation for Safe hash reconstruction

Impact:	Warning	**Likelihood:**	N/A
Type:	Data validation	**Discoverer:**	Human
Target:	CoSignerPolicy.sol, IncreasedThresholdPolicy.sol, SafenetPolicy.sol, SafePolicyGuard.sol		

Description

`CoSignerPolicy`, `IncreasedThresholdPolicy`, and `SafenetPolicy` rebuild the top-level Safe transaction hash with zero values for `safeTxGas`, `baseGas`, `gasPrice`, `gasToken`, and `refundReceiver`. The `CoSignerPolicy` call is representative:

Listing 22. Excerpt from CoSignerPolicy.checkTransaction

```
53 bytes32 safeTxHash = ISafe(safe).getTransactionHash(
54     to,
55     value,
56     data,
57     operation,
58     // TODO(nlordell): We assume 0 values here - they aren't available in
   the policy engine.
59     // See the `PolicyEngine` contract for more details and rationale.
60     0,
61     0,
62     0,
63     address(0),
64     address(0),
```

However, `SafePolicyGuard` only enforces zero values for `gasPrice` and `safeTxGas`. `baseGas`, `gasToken`, and `refundReceiver` are accepted by the Safe entry point but discarded before policy checks:

Listing 23. Excerpt from SafePolicyGuard.checkTransaction

```
214 // Gas refunds are deliberately unsupported: they are another way to move
```

```
    funds out of a
215 // Safe, are rarely used, and are not worth covering in the policy
    interface. With
216 // `gasPrice == 0` the Safe skips `handlePayment` entirely, so `baseGas`,
    `gasToken` and
217 // `refundReceiver` cannot move value and need no policy check.
218 require(gasPrice == 0, NonZeroGasPrice());
219
220 // A non-zero `safeTxGas` lets a Safe transaction whose inner call fails
    complete without
221 // reverting. Once policy checks can mutate state, that would commit any
    state a policy
222 // staged during the pre-check against a failed action. Forbid it so a
    failed execution
223 // always reverts atomically, keeping policy state in sync with execution
    outcome.
224 require(safeTxGas == 0, NonZeroSafeTxGas());
```

If any unchecked field is non-zero, the policy computes a hash that does not match the transaction hash signed by standard Safe tooling. The current result is fail-closed because `gasPrice == 0` disables refund payment, but these policies reject otherwise valid transactions and depend on an undocumented coupling between the guard and hash reconstruction.

Recommendation

Enforce all five gas/refund fields as zero in `SafePolicyGuard`, or pass the real top-level values through to policies. Keep an explicit zero convention for `MultiSendPolicy` sub-transaction checks, which do not have independent gas/refund fields. Update the reconstruction comments to state the enforced invariants and remove the stale `PolicyEngine` reference.

Acknowledgment 1.1

The finding was acknowledged and deferred to a future iteration.

Go back to Findings Summary

W7: Selector-zero access key ambiguity

Impact:	Warning	**Likelihood:**	N/A
Type:	Configuration	**Discoverer:**	AI
Target:	PolicyEngine.sol, AccessSelector.sol		

Description

`PolicyEngine._decodeSelector` maps empty calldata to `bytes4(0)`. It also returns `bytes4(data)` for calldata with at least four bytes, so calldata beginning with `0x00000000` has the same decoded selector.

Listing 24. Excerpt from PolicyEngine

```
247 function _decodeSelector(bytes calldata data) internal pure returns
    (bytes4 selector) {
248     if (data.length >= 4) {
249         return bytes4(data);
250     } else if (data.length == 0) {
251         return bytes4(0);
```

`AccessSelector.getSelector` exposes only the selector field from the packed access key, so policies using the access key cannot distinguish empty calldata from selector-zero calldata. Executing a fallback policy also passes an access key with zero selector and target address, because fallback access keys set only the operation field.

Listing 25. Excerpt from AccessSelector

```
38 function createFallback(Operation operation) internal pure returns (T) {
39     return T.wrap(uint256(operation) << 216);
```

Listing 26. Excerpt from AccessSelector

```
56 function getSelector(T self) internal pure returns (bytes4) {
```

```
57     return bytes4(bytes32(T.unwrap(self)));
```

This does not authorize arbitrary selectors, and the selected policy still receives the full calldata. However, integrations that treat selector zero as bare native transfer semantics may unintentionally allow additional recipient behavior unless the policy validates `data.length == 0`.

Recommendation

Document that selector zero is not an empty-calldata marker. Policies intended to allow only empty calldata should explicitly require `data.length == 0`.

Policies intended to operate on the transaction target should use `to` instead of `access.getTarget()` in `checkTransaction`.

Fix 1.1

The selector-zero ambiguity was documented, and `NativeTransferPolicy` was updated to require empty calldata.

Go back to Findings Summary

W8: Unbounded epoch rollover

Impact:	Warning	**Likelihood:**	N/A
Type:	Data validation	**Discoverer:**	Human
Target:	EpochRollover.sol		

Description

`EpochRollover.rollover` only checks that `proposedEpoch` is greater than `parentEpoch`. It does not cap the epoch or require a one-epoch advance.

Listing 27. Excerpt from EpochRollover.rollover

```
116 require(proposedEpoch > parentEpoch, EpochNotAdvancing());
```

Accepted epoch entries are appended permanently.

Listing 28. Excerpt from EpochRollover.rollover

```
124 self.entries[newGroupKey.x][newGroupKey.y][proposedEpoch] = true;
```

A valid validator-set signature is still required, so this is primarily a defense-in-depth issue against malformed signed rollover messages. If such a rollover records `type(uint64).max`, that `(groupKey, epoch)` can never be used as a parent because no greater `uint64` exists. A large non-maximum jump can also move the execution-chain forest ahead of the consensus epoch, causing legitimate sequential rollovers from that branch to revert with `EpochNotAdvancing()` until the off-chain epoch catches up.

Because `SafenetPolicy` has no on-chain revocation or re-seeding path for `$epochs`, recovery requires replacing and reconfiguring the policy.

Recommendation

Reject `proposedEpoch == type(uint64).max` and cap `proposedEpoch - parentEpoch` according to the expected rollover cadence. If larger jumps must remain valid, add an explicit revocation or re-seeding mechanism and monitor `EpochRolledOver` events with unexpected epoch deltas.

Acknowledgment 1.1

The finding was acknowledged.

Go back to Findings Summary

W9: Retained policy grants after detachment

Impact:	Warning	**Likelihood:**	N/A
Type:	Configuration	**Discoverer:**	AI
Target:	PolicyEngine.sol, AllowedModulePolicy.sol		

Description

`PolicyEngine._confirmPolicy` updates the access-selector mapping and only calls `configure` for the new nonzero policy. Detaching or replacing a policy therefore does not clear state stored inside the previous policy.

Listing 29. Excerpt from PolicyEngine._confirmPolicy

```
288 _updatePolicy(safe, access, policy);
289
290 // Configuring policy
291 if (policy != address(0)) {
292     require(IPolicy(policy).configure(safe, access, data),
    PolicyConfigurationFailed());
```

`AllowedModulePolicy` stores grants by `(policyGuard, safe, module)` and reuses that storage when the same guard/Safe namespace is used again. Its checks consume the retained grant directly, so reattaching the policy can make previously configured modules effective again unless each grant was explicitly revoked.

Listing 30. Excerpt from AllowedModulePolicy

```
16 mapping(address policyGuard => mapping(address safe => mapping(address
   module => bool allowed)))
17     private $allowedModules;
```

Listing 31. Excerpt from AllowedModulePolicy.checkTransaction

```
46 require($allowedModules[msg.sender][safe][module], UnauthorizedModule());
```

Listing 32. Excerpt from AllowedModulePolicy.configure

```
65 $allowedModules[msg.sender][safe][module] = allowed;
```

This persistence is auditable and documented, and does not create an unauthorized configuration path by itself. The operational risk is treating policy detachment, replacement, or guard reinstallation as a full configuration reset while old allowlist entries remain active.

Recommendation

Consider adding an explicit reset operation that increments a configuration epoch per (`guard`, `safe`) and includes that epoch in resettable policy-grant storage keys. If the reset is intended to cover the whole Safe configuration, version the engine policy mapping as well.

Keep replay-protection state, such as spent signatures, outside the resettable grant namespace. Otherwise, document that detachment, replacement, and guard reinstallation do not reset grants and require explicit revocations.

Acknowledgment 1.1

The finding was acknowledged and deferred to a future iteration.

Go back to Findings Summary

I1: Unsupported high-x group keys

Impact:	Info	**Likelihood:**	N/A
Type:	Code quality	**Discoverer:**	AI
Target:	EpochRollover.sol, Secp256k1.sol		

Description

`EpochRollover` accepts genesis and successor group keys after only `Secp256k1.requireNonZero`. That helper checks that coordinates are field elements on the curve; it does not require `x < N`.

Listing 33. Excerpt from EpochRollover

```
84 function initialize(T storage self, uint64 genesisEpoch, Secp256k1.Point
   memory genesisKey) internal {
85     Secp256k1.requireNonZero(genesisKey);
86     self.entries[genesisKey.x][genesisKey.y][genesisEpoch] = true;
```

Listing 34. Excerpt from EpochRollover.rollover

```
115 require(isKnown(self, parentKey, parentEpoch), UnknownParent());
116 require(proposedEpoch > parentEpoch, EpochNotAdvancing());
117 Secp256k1.requireNonZero(newGroupKey);
118
119 bytes32 message =
120     ConsensusMessages.epochRollover(domainSeparator, parentEpoch,
    proposedEpoch, rolloverBlock, newGroupKey);
121 FROST.verify(parentKey, signature, message);
```

Listing 35. Excerpt from Secp256k1

```
181 function requireNonZero(Point memory p) internal pure {
182     require(_satisfiesCurveEquation(p.x, p.y), NotOnCurve());
183 }
```

However, `FROST.verify` uses `Secp256k1.mulmuladd`, which passes the group

key's x coordinate as the ECDSA r input to ecrecover. ECDSA rejects r >= N, so a valid curve point with N <= x < P can be recorded but cannot verify attestations or future rollovers.

Listing 36. Excerpt from Secp256k1.mulmuladd

```
147 // Perform `ecrecover(z * Px, v, Px, e * Px) = address(-R)`
148 let ptr := mload(0x40)
149 mstore(ptr, mulmod(z, px, N))
150 mstore(add(ptr, 0x20), add(and(py, 1), 27))
151 mstore(add(ptr, 0x40), px)
152 mstore(add(ptr, 0x60), mulmod(e, px, N))
153 let minusR :=
154     mul(mload(0x00), and(eq(returndatasize(), 0x20), staticcall(gas(),
    0x1, ptr, 0x80, 0x00, 0x20)))
```

Under the uniform-x approximation, this unsupported subset has probability (P - N) / P = 3.734455345040134e-39, or about one in 2.677766655660072e38, for a randomly generated key.

Recommendation

Document that curve validity alone does not guarantee verifier compatibility. Optionally reject group keys with x >= Secp256k1.N in initialize and rollover, and update the vendored mulmuladd comment to use the corrected unsupported-key probability of about one in 2.678e38.

Acknowledgment 1.1

The finding was acknowledged, and the high-x key limitation was documented in NatSpec.

Go back to Findings Summary

I2: Unused `InvalidSelector` error

Impact:	Info	**Likelihood:**	N/A
Type:	Unused code	**Discoverer:**	Human
Target:	CoSignerPolicy.sol		

Description

`CoSignerPolicy` declares `InvalidSelector`, but never reverts with it. This makes the published ABI include an error that cannot be emitted, which can mislead integrators and reviewers enumerating the contract's failure modes.

Listing 37. Excerpt from CoSignerPolicy

```
34 error InvalidSelector();
```

Recommendation

Delete the unused `InvalidSelector` declaration.

Fix 1.1

The unused error definition was removed.

Go back to Findings Summary

I3: Missing nonzero FROST identifier check

Impact:	Info	**Likelihood:**	N/A
Type:	Standards violation	**Discoverer:**	AI
Target:	FROST.sol		

Description

`FROST.identifier` returns the derived hash-to-field value without enforcing the RFC 9591 `NonZeroScalar` participant identifier invariant.

Listing 38. Excerpt from FROST.identifier

```
80 return _hid(abi.encodePacked(participant));
```

The derivation reduces the expanded hash modulo `Secp256k1.N`, so `0` remains a possible scalar-field result.

Listing 39. Excerpt from FROST

```
345 bytes memory uniform = _expandMessageXmd(message, dst, 48);
346 uint256 n = Secp256k1.N;
347 assembly ("memory-safe") {
348     e := mulmod(mload(add(uniform, 0x20)),
    0x100000000000000000000000000000000, n)
349     e := addmod(e, shr(128, mload(add(uniform, 0x40))), n)
350 }
```

In Shamir secret sharing, evaluation at `0` is the shared secret itself, so a zero participant identifier must not be accepted. This is a vendored-library correctness issue: current production verification does not call `identifier`, and `_encodeCommitments` independently rejects zero through its strict `id > last` ordering check.

Recommendation

Import the upstream `assert(id != 0)` after deriving the identifier, and document that a successful `FROST.identifier` return is nonzero.

Fix 1.1

The finding was fixed by following the recommendation.

Go back to Findings Summary

I4: Inaccurate NatSpec comments

Impact:	Info	**Likelihood:**	N/A
Type:	Code quality	**Discoverer:**	AI
Target:	AccessSelector.sol, ISafe.sol, Secp256k1.sol		

Description

Several NatSpec comments do not match the implementation they document. This has no runtime impact, but can mislead integrators and generated documentation.

`AccessSelector.getOperation` documents non-existent `OP_CALL` and `OP_DELEGATECALL` names instead of `Operation.CALL` and `Operation.DELEGATECALL`.

Listing 40. Excerpt from AccessSelector.getOperation

```
63 * @return The operation type (OP_CALL or OP_DELEGATECALL)
```

`ISafe.getStorageAt` describes `length` as bytes in the notice, while the parameter is the number of 32-byte storage words.

Listing 41. Excerpt from ISafe.getStorageAt

```
97 * @notice Reads `length` bytes of storage in the current contract
98 * @param offset - the offset in the current contract's storage in words to
   start reading from
99 * @param length - the number of words (32 bytes) of data to read
```

`ISafe.execTransaction` has an extra brace and documents an unconditional `gasPrice * gasLimit` refund, which does not match Safe 1.5.0 refund behavior.

Listing 42. Excerpt from ISafe.execTransaction

```
104 /** @notice Executes a `operation` {0: Call, 1: DelegateCall}} transaction
    to `to` with `value` (Native Currency)
105  *          and pays `gasPrice` * `gasLimit` in `gasToken` token to
    `refundReceiver`.
106  * @dev The fees are always transferred, even if the user transaction
    fails.
```

Secp256k1.add reports an outdated MODEXP gas estimate for the _divmod precompile call.

Listing 43. Excerpt from Secp256k1.add

```
78 *      (one per operation), as that is by far the most expensive
   computation (~200 gas for the `modexp` precompile
79 *      call).
```

Secp256k1.add documents the final else branch as the P = -P case. The branch is reached when px == qx && py != qy, which for two on-curve points means qy = P - py, i.e. Q = -P. P = -P would require py == 0, and no such point exists on secp256k1.

Listing 44. Excerpt from Secp256k1.add

```
106 } else {
107     // This branch happens iff `P = -P` in which case their sum is the
    point at infinity (represented by the
108     // 0-value) that `r` is initialized with.
109     return r;
```

Recommendation

Update the comments to use the actual Operation members, document getStorageAt.length as 32-byte words, describe the Safe refund calculation and conditions, remove the extra brace, remove or correct the MODEXP gas estimate, and change the Secp256k1.add else branch comment to state that it handles Q = -P.

Fix 1.1

The documentation comments were fixed.

Go back to Findings Summary

I5: Undocumented co-signature format differences

Impact:	Info	**Likelihood:**	N/A
Type:	Code quality	**Discoverer:**	AI
Target:	CoSignerPolicy.sol		

Description

`CoSignerPolicy` passes the reconstructed `Safe` transaction hash and `context` bytes directly to OpenZeppelin `SignatureChecker`. For EOA co-signers, this accepts a narrower format than `Safe` owner signatures, while unsupported encodings fail with the same `Unauthorized()` error as a wrong signer.

Listing 45. Excerpt from CoSignerPolicy.checkTransaction

```
72 bool validSignature = SignatureChecker.isValidSignatureNow(cosigner,
   safeTxHash, context);
73 require(validSignature, Unauthorized());
```

The differences are:

- EOA policy signatures are checked directly against `safeTxHash`, while `Safe` also supports the `eth_sign` form over the Ethereum signed-message-prefixed hash;
- EOA policy signatures must use `v` equal to `27` or `28`, while `Safe` encodes the `eth_sign` form with `v` equal to `31` or `32`;
- EOA policy signatures must satisfy OpenZeppelin `ECDSA` low-`s` checks, while `Safe` accepts high-`s` values accepted by `ecrecover`;
- contract co-signer signatures are passed as raw ERC-1271 payloads, while `Safe` wraps contract signatures in an aggregate owner-signature record; and
- `CoSignerPolicy` does not support the `Safe` approved-hash signature form.

As a result, integrators who reuse `Safe` owner-signature encodings as the policy `context` can have otherwise owner-authorized transactions revert at the co-signer check. This fails closed and does not create a replay issue, but the accepted signing format should be explicit.

Recommendation

Document the differences between `CoSignerPolicy` co-signatures and `Safe` owner signatures. Where practical, consider unifying the behavior with `Safe`; otherwise, document the unsupported formats and the expected `context` encoding for EOA and contract co-signers.

Fix 1.1

The signature verification differences were documented.

Go back to Findings Summary

I6: Incorrect guard-removal documentation and test

Impact:	Info	**Likelihood:**	N/A
Type:	Code quality	**Discoverer:**	AI
Target:	README.md, safePolicyGuard.spec.ts		

Description

The README documents an atomic guard-removal flow where a matured configuration is applied and used to remove the guard in the same `MultiSend` batch.

Listing 46. Excerpt from README

```
177 To remove a guard, instead of baking in the delay mechanism within the
    guard contract, we use the delay mechanism which is already present for
    any policy to get activated. To remove a guard:
178 - We `requestConfiguration(...)` with the `configureRoot` as the data with
    [AllowPolicy](./contracts/policies/AllowPolicy.sol) and selector as
    `setGuard(...)`, target as Safe itself, and operation as `CALL`
179 - Once the delay is over, we can apply the policy using
    `applyConfiguration(...)` and also remove the Guard (we can use MultiSend
    for the same to do in a single transaction).
```

That sequence is not valid when the outer batch is protected by `MultiSendPolicy`. The guard validates the Safe transaction before the batch executes, and `MultiSendPolicy` recursively validates each child during that pre-execution phase. At that point, the `applyConfiguration` child has only been checked; it has not executed `_confirmPolicy`. The later `setGuard` child is therefore evaluated against the previous policy mapping and is denied unless guard removal was already configured.

The regression test does not cover this path because it authorizes the outer `multiSend` selector with unrestricted `AllowPolicy`, not `MultiSendPolicy`.

Listing 47. Excerpt from safePolicyGuard

```
208 createConfiguration({
209   target: await multiSend.getAddress(),
210   selector: multiSend.interface.getFunction('multiSend')?.selector,
211   operation: SafeOperation.DelegateCall,
212   policy: await allowPolicy.getAddress()
213 })
```

The same test then places activation and guard removal in one batch.

Listing 48. Excerpt from safePolicyGuard

```
238 buildSafeTransaction({
239   to: await safePolicyGuard.getAddress(),
240   data: safePolicyGuard.interface.encodeFunctionData('applyConfiguration',
    [configurations]),
241   nonce: 0
242 }),
243 buildSafeTransaction({
244   to: await safe.getAddress(),
245   data: safe.interface.encodeFunctionData('setGuard', [ZeroAddress]),
246   nonce: 1
```

Because the outer batch is already allowed, child transactions are not recursively checked. The test only proves that a delayed configuration can be applied inside an already authorized batch; it does not prove that the newly applied guard-removal permission can be used atomically. This fails closed for recursively checked batches, but the documentation and test describe unsupported behavior.

Recommendation

Document guard removal as two Safe executions: first apply the matured configuration, then execute `setGuard` or `setModuleGuard` using the now-active policy. Update the regression test to configure `MultiSendPolicy` for the outer batch and assert that activation-and-removal in one recursively checked batch fails without pre-existing removal permission, while removal succeeds after the matured configuration was applied in an earlier Safe execution.

Fix 1.1

The guard removal flow was corrected in `README.md`.

Go back to Findings Summary

I7: Fallback authorization after exact policy clearing

Impact:	Info	**Likelihood:**	N/A
Type:	Configuration	**Discoverer:**	AI
Target:	README.md, SafePolicyGuard.sol		

Description

The README and configuration NatSpec do not distinguish removing an exact policy override from explicitly denying the matching calls. `_confirmPolicy` stores the supplied `policy` value, and `address(0)` is not configured as a policy.

Listing 49. Excerpt from PolicyEngine._confirmPolicy

```
287 // Update the policy mapping
288 _updatePolicy(safe, access, policy);
289
290 // Configuring policy
291 if (policy != address(0)) {
292     require(IPolicy(policy).configure(safe, access, data),
    PolicyConfigurationFailed());
```

`getPolicy` treats a zero policy entry as absent and resolves through the operation fallback.

Listing 50. Excerpt from PolicyEngine.getPolicy

```
134 // Use the fallback policy for the given operation if there is no specific
    one.
135 if (policy == address(0)) {
136     access = AccessSelector.createFallback(operation);
137     policy = policies[access];
```

As a result, setting an exact entry to `address(0)` restores fallback authorization for that selector. If the fallback is more permissive, policy-governed calls that were previously restricted by the exact entry can

become allowed by the fallback; if no fallback exists, ordinary calls remain denied. This is expected lookup behavior after an authorized configuration change, but integrators may mistake clearing for revocation instead of installing `DenyPolicy`.

Recommendation

Document that setting an exact policy entry to `address(0)` removes the override and restores operation fallback resolution. State that `DenyPolicy` should be configured on the exact entry when matching calls must be denied regardless of the fallback, and add regression coverage for clearing while a fallback is configured.

Fix 1.1

The behavior was documented in code and in `README.md`.

Go back to Findings Summary

I8: Ambiguous policy replacement events

Impact:	Info	**Likelihood:**	N/A
Type:	Logging	**Discoverer:**	AI
Target:	PolicyEngine.sol		

Description

`PolicyEngine._confirmPolicy` overwrites the policy mapping before emitting `PolicyConfirmed`. The event only reports the new `policy` and configuration data, so an event by itself cannot distinguish an initial assignment or reconfiguration from a replacement, nor identify the displaced policy.

Listing 51. Excerpt from PolicyConfirmed

```
110 event PolicyConfirmed(
111     address indexed safe,
112     address indexed target,
113     bytes4 selector,
114     Operation operation,
115     address policy,
116     bytes data
117 );
```

Listing 52. Excerpt from PolicyEngine._confirmPolicy

```
288 _updatePolicy(safe, access, policy);
```

Listing 53. Excerpt from PolicyEngine._confirmPolicy

```
295 emit PolicyConfirmed(safe, target, selector, operation, policy, data);
```

As a result, monitoring tools must reconstruct prior mapping state to detect replacements. This is an observability gap only; it does not bypass policy authorization.

Recommendation

Store the previous policy before `_updatePolicy`. Emit a dedicated `PolicyReplaced` event for replacements with the Safe, access selector details, previous policy, and new policy, while continuing to emit `PolicyConfirmed` for configuration data.

Acknowledgment 1.1

The finding was acknowledged and deferred to a future iteration.

Go back to Findings Summary

Report Revision 1.1

Revision Team

NAME	POSITION
Michal Převrátil	Lead Auditor
Josef Gattermayer, Ph.D.	Audit Supervisor

System Overview

The follow-up changes restricted `NativeTransferPolicy` to plain native transfers, made `CoSignerPolicy` reject module transactions, and added an explicit policy guard argument to `getCoSigner`. The vendored Safenet cryptography libraries were synchronized with upstream, and the documentation and tests were updated. The Policy Engine's policy selection and transaction routing remained unchanged.

Trust Model

The trust model remained unchanged from the previous revision. Safe owners still controlled guard and policy configuration, while enabled modules could move funds only through paths permitted by active policies. `CoSignerPolicy` now rejected module transactions, and `NativeTransferPolicy` approved only plain native transfers.

Appendix A: How to cite

Cite this document as:

> ack3, Security Review | Safe: Policy Engine, November 9, 2026.



Thank you.

COMPANY	ACK3 LTD
WEB	ack3.ai
X	@ack3ai
MAIL	hello@ack3.ai